

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import AutoMinorLocator
import sklearn as sk
from scipy import stats as st
from scipy.optimize import minimize
import pandas as pd
```

```
In [2]: np.set_printoptions(formatter={'float': lambda x: "{0:0.3e}".format(x)}) #to
```

Functions

```
In [3]: #for plotting distributions
def plot_distrib(data, bins = 100, title = ""):
    plt.hist(data, bins=bins, density=True)
    plt.title(title)
    plt.ylabel("Probability density")
```

```
In [4]: #to calculate 1σ uncertainty bands and medians
def median_and_error(data):
    med, low, high = np.percentile(data, [50, 16, 84])
    median_and_error = np.array([med, low - med, high - med])

    return median_and_error
```

```
In [5]: #Fits a skew-normal distribution to a given set of percentiles.
def fit_skewnorm(percentiles, values): #the percentiles and corresponding va

    #set initial guesses as if it was a gaussian
    median = np.interp(0.5, percentiles, values)
    p10 = np.interp(0.1, percentiles, values)
    p90 = np.interp(0.9, percentiles, values)
    init_scale = (p90 - p10) / 2.56 if (p90 - p10) > 0 else 1 #for a gaussi

    initial_guesses = [0., median, init_scale]

    bounds = [(-20, 20), (None, None), (1e-6, None)]  #(a, loc, scale) -> kee

    #Error function to minimize
    def error_function(params):
        a, loc, scale = params
        theoretical_values = st.skewnorm.ppf(percentiles, a=a, loc=loc, scal
        return np.sum((theoretical_values - values) ** 2) #sum of squared er

    result = minimize(
        error_function,
        initial_guesses,
        method='SLSQP',
        bounds=bounds,
        options={'ftol': 1e-9, 'disp': False}
    )
```

```
return result.x #(a, loc, scale)
```

Plotting isolation time distributions for the SLR p-isotopes

This notebook calculates and plots isolation time distributions for ^{92}Nb and ^{146}Sm using abundance ratios from the CCSN models from [Ritter et al. \(2018\)](#) and [Roberti et al. \(2024\)](#) and the SNIa ratios from [Lugaro et al. \(2016\)](#). This work is largely based on re-implementing the *Mathematica* notebook 'isolation_time_figures.nb' from [Leckenby et al. \(2024\)](#) in *Python* and expanding its functionality to account for mean-life uncertainties and modeling the GCE stochastic uncertainties described in [Côte et al. \(2019\)](#) by fitting a skew-normal distribution to the medians and confidence intervals they provide.

```
In [6]: k = 2.3 #best
```

```
In [7]: #production ratio
prIaNb = 1.58*10**-3
prIaSm = 0.347

EssNb = (3.2 * 10**-5, 0.3 * 10**-5)
EssSm = (8.28 * 10**-3, 0.44 * 10**-3)

#mean-life
tauNb = np.array([34.7, 2.4]) / np.log(2.) #from NUBASE2020
tauSm = np.array([71.74, 7.39]) / np.log(2.) #from Tang et al. (2025)
tau = np.array([tauNb, tauSm])

#age of the galaxy
TGal = 8400
```

```
In [8]: #τ for table
print(tau)
```

```
[5.006e+01 3.462e+00]
[1.035e+02 1.066e+01]
```

Fit skewnormal distribution to percentiles

Assuming that a skew-normal distribution is a sufficient approximation for the assymmetric distributions of the ISS ratios.

Percentiles from Table 3 in [Côté et al. \(2019\)](#), γ is taken to be 10 Myr.

```
In [9]: percentiles = [0.16, 0.50, 0.84]

statNb = [3.13-0.73, 3.13, 3.13+0.80]
statSm = [9.99-0.84, 9.99, 9.99+0.87]
```

```

paramNb = fit_skewnorm(percentiles, statNb)
paramSm = fit_skewnorm(percentiles, statSm)

distNb = st.skewnorm(paramNb[0], paramNb[1], paramNb[2])
distSm = st.skewnorm(paramSm[0], paramSm[1], paramSm[2])

#normalise
paramNb[1:] = paramNb[1:] / distNb.mean()
paramSm[1:] = paramSm[1:] / distSm.mean()

#create normalised distribution
distNb = st.skewnorm(paramNb[0], paramNb[1], paramNb[2])
distSm = st.skewnorm(paramSm[0], paramSm[1], paramSm[2])

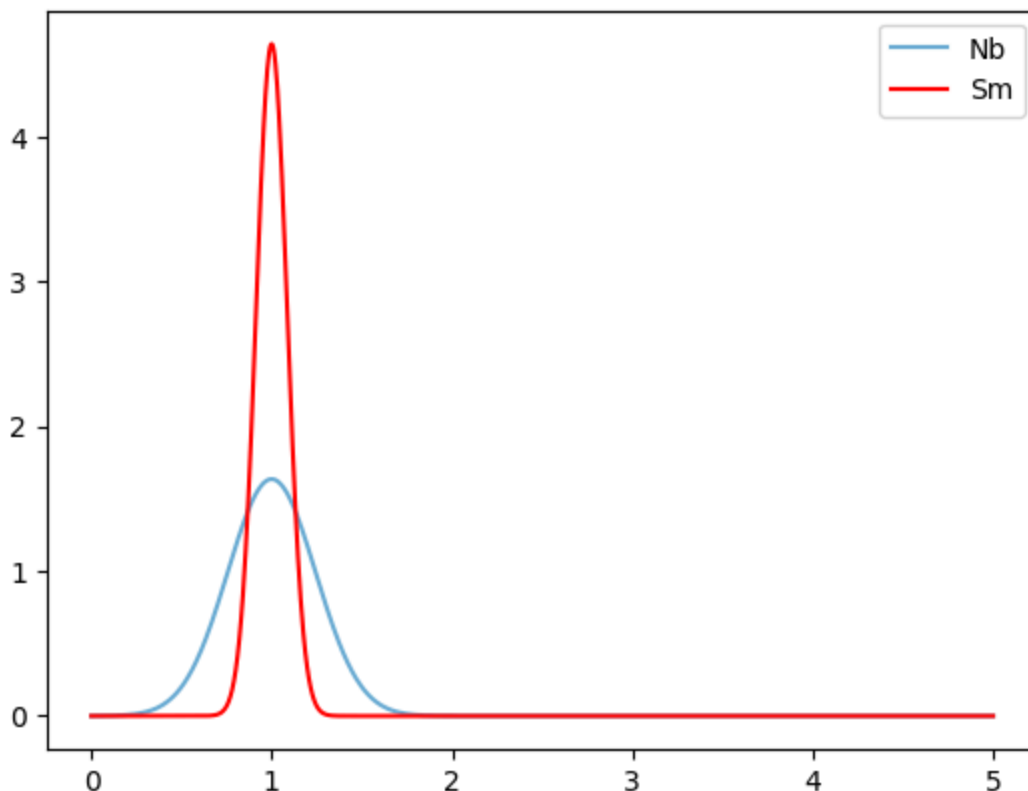
```

```

In [10]: #visualising the fitted distributions
x = np.linspace(0,5,1000)
plt.plot(x,distNb.pdf(x=x),marker='none',label='Nb',color=plt.get_cmap('Blue')
plt.plot(x,distSm.pdf(x=x),marker='none',label='Sm',color='r')
plt.legend()

```

Out[10]: <matplotlib.legend.Legend at 0x77ce40a983e0>



Calculate and plot isolation times for Typela prodction

Then I draw Monte Carlo distributions of the ESS and ISM ratios, and mean-lives. τ and ESS is assumed to be Gaussian, ISM comes from the fitted skew-normal distributions.

```

In [11]: mcNum = 10**7 #number of samples for MC

In [12]: #Sampling distributions for the  $\tau$  values
ratTauNb = np.clip(np.random.normal(loc = tauNb[0], scale = tauNb[1], size=n
ratTauSm = np.clip(np.random.normal(loc = tauSm[0], scale = tauSm[1], size=n

In [13]: #The ISM ratios
IsmNb = k * prIaNb * ratTauNb / TGal
IsmSm = k * prIaSm * ratTauSm / TGal

In [14]: #Sampling distributions for the ISM values
ratIsmNb = np.clip(IsmNb * distNb.rvs(mcNum), 1e-30, np.inf)
ratIsmSm = np.clip(IsmSm * distSm.rvs(mcNum), 1e-30, np.inf)

In [15]: #Sampling normal distribution for the ESS values
ratEssNb = np.clip(np.random.normal(loc = EssNb[0], scale = EssNb[1], size=n
ratEssSm = np.clip(np.random.normal(loc = EssSm[0], scale = EssSm[1], size=n

In [16]: #Calculating isolation time
isoTNb = -np.log(ratEssNb / ratIsmNb) * ratTauNb
isoTSm = -np.log(ratEssSm / ratIsmSm) * ratTauSm

In [17]: #downsample for visualisation (this is for optimisation)
isoTSampleNb = sk.utils.resample(isoTNb, n_samples=1000000)
isoTSampleSm = sk.utils.resample(isoTSm, n_samples=1000000)

In [18]: #create the approximate isolation time distributions
isoTSampleNbKern = sk.neighbors.KernelDensity(kernel='gaussian', bandwidth=1
isoTSampleNbKern.fit(isoTSampleNb.reshape(-1, 1))
isoTSampleSmKern = sk.neighbors.KernelDensity(kernel='gaussian', bandwidth=1
isoTSampleSmKern.fit(isoTSampleSm.reshape(-1, 1))

Out[18]:
KernelDensity ⓘ ⓘ
Parameters

In [19]: x = np.linspace(-120, 120, 100).reshape(-1, 1)

In [20]: #evaluate the distribution at 100 points
isoTSampleNbDensity = np.exp(isoTSampleNbKern.score_samples(x))
isoTSampleSmDensity = np.exp(isoTSampleSmKern.score_samples(x))

In [21]: #Creating figure
blue = plt.get_cmap('Blues')(2/4)

plt.plot(x, isoTSampleNbDensity, color=blue, label='$^{92}$Nb')
plt.fill_between(x.flatten(), isoTSampleNbDensity, 0, color=blue, alpha=0.3

plt.plot(x, isoTSampleSmDensity, color='red', label='$^{146}$Sm')
plt.fill_between(x.flatten(), isoTSampleSmDensity, 0, color='red', alpha=0.

```

```

plt.xlabel('Isolation time (Myr)')
plt.ylabel('Probability density')
plt.legend(loc='upper right')

#for tickmarks
plt.gca().xaxis.set_minor_locator(AutoMinorLocator())
plt.gca().yaxis.set_minor_locator(AutoMinorLocator())
plt.gca().tick_params(axis='x', which='both', top=True)
plt.gca().tick_params(axis='y', which='both', right=True)

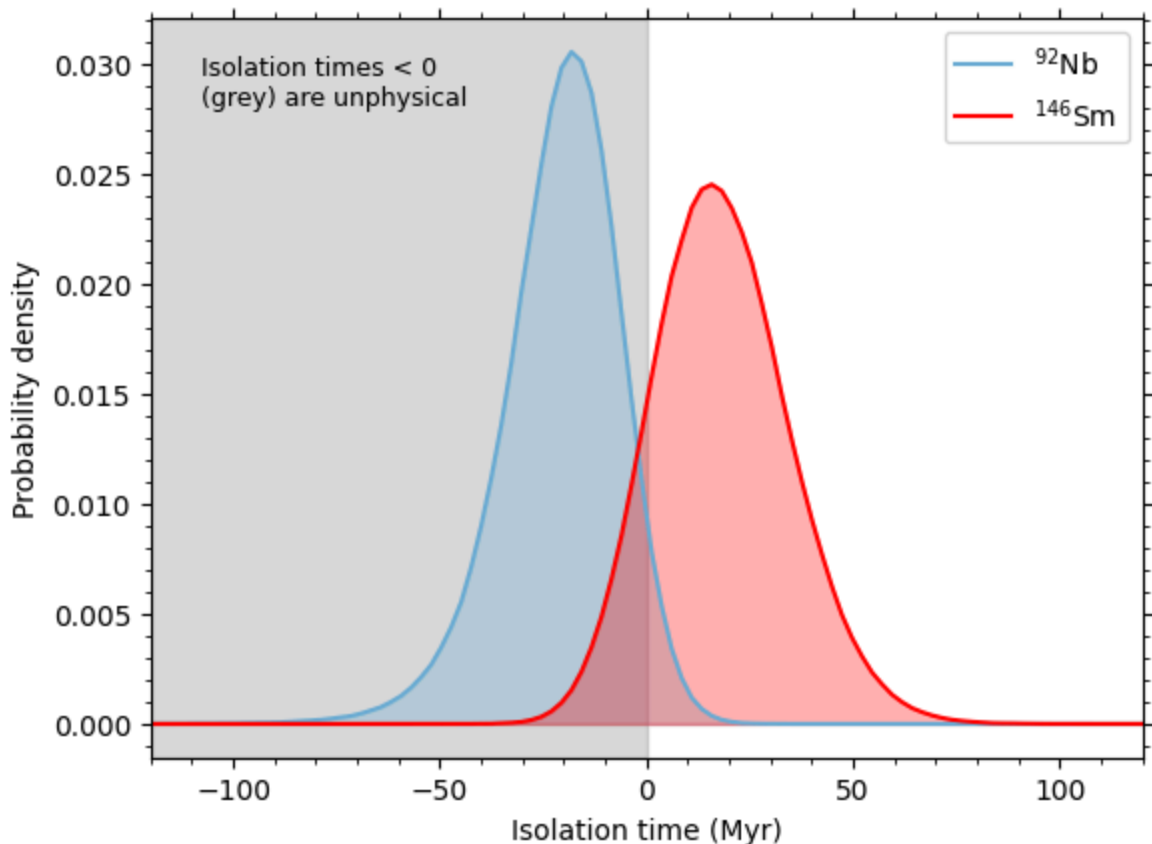
plt.legend(loc='upper right')

#for the gray box
plt.axvspan(xmin=x.flatten()[0], xmax=0, color='gray', alpha=0.3, zorder=0)
plt.text(0.05, 0.95, "Isolation times < 0 \n(grey) are unphysical", transform=plt.gca().transAxes)

plt.xlim(x[0], x[-1])

plt.savefig("../figures/"+"isoT_Ia.png", bbox_inches='tight')
plt.show()

```



Calculate and plot isolation times for CCSN production

```

In [22]: # Import production ratios of different CCSN models
data = pd.read_csv("../data/production_ratios.txt", sep='\\s+')
data = data.set_index("model")

```

```

In [23]: shock = np.array(data['initial_shock'])
         mtot = np.array(data['MTOT'])
         mrem = np.array(data['MREM'])
         msurf = np.array(data['MSURF'])
         modelSet = np.array(data['Set'])

         prCCSnNb = np.array(data['Nb-92/Mo-92'])
         prCCSnSm = np.array(data['Sm-146/Sm-144'])

In [24]: mcNum = 10**7 #number of samples for MC

In [25]: #Sampling distributions for the  $\tau$  values
         ratTauNb = np.clip(np.random.normal(loc=tauNb[0], scale=tauNb[1], size=mcNum)
         ratTauSm = np.clip(np.random.normal(loc=tauSm[0], scale=tauSm[1], size=mcNum)

In [26]: #The ISM ratios
         IsmNbCCSn = k * prCCSnNb[:, np.newaxis] * ratTauNb / TGal
         IsmSmCCSn = k * prCCSnSm[:, np.newaxis] * ratTauSm / TGal

In [27]: ratIsmNbCCSn = np.clip(IsmNbCCSn * distNb.rvs(mcNum), 1e-30, np.inf)
         ratIsmSmCCSn = np.clip(IsmSmCCSn * distSm.rvs(mcNum), 1e-30, np.inf)

         isoTNbCCSn = -np.log(ratEssNb / ratIsmNbCCSn) * ratTauNb
         isoTsmCCSn = -np.log(ratEssSm / ratIsmSmCCSn) * ratTauSm

In [28]: isoTSampleNbCCSn = []
         isoTSampleSmCCSn = []

         for i in range(np.size(prCCSnNb)):
             isoTSampleNbCCSn.append(sk.utils.resample(isoTNbCCSn[i], n_samples=10000)
             isoTSampleSmCCSn.append(sk.utils.resample(isoTsmCCSn[i], n_samples=10000)

In [29]: isoTNbKernCCSn = []
         isoTsmKernCCSn = []

         for i in range(np.size(prCCSnNb)):
             isoTNbKernCCSn.append(sk.neighbors.KernelDensity(kernel='gaussian', band
             isoTNbKernCCSn[i].fit(isoTSampleNbCCSn[i].reshape(-1, 1))
             isoTsmKernCCSn.append(sk.neighbors.KernelDensity(kernel='gaussian', band
             isoTsmKernCCSn[i].fit(isoTSampleSmCCSn[i].reshape(-1, 1))

In [30]: #create the ranges for plotting the data (limits selected based on previous
         limits = np.array([[-150,50],[-150,50],[-150,110],[-150,110],[-380,110]])
         x = np.linspace(limits[:,0], limits[:,1], 1000).T

In [31]: #evaluate the distributions at 1000 points
         isoTNbDensityBest_CCSn = []
         isoTsmDensityBest_CCSn = []

         for i in range(np.size(prCCSnNb)):
             isoTNbDensityBest_CCSn.append(np.exp(isoTNbKernCCSn[i].score_samples(x[i]
             isoTsmDensityBest_CCSn.append(np.exp(isoTsmKernCCSn[i].score_samples(x[i]

```

```

In [32]: blue = plt.get_cmap('Blues')(2/4)

for i in range(np.size(prCCSnNb)):

    plt.plot(x[i], isoTNbDensityBest_CCSn[i], color=blue, label='${92}$Nb')
    plt.fill_between(x[i], isoTNbDensityBest_CCSn[i], 0, color=blue, alpha=

    #for tickmarks
    plt.gca().xaxis.set_minor_locator(AutoMinorLocator())
    plt.gca().yaxis.set_minor_locator(AutoMinorLocator())
    plt.gca().tick_params(axis='x', which='both', top=True)
    plt.gca().tick_params(axis='y', which='both', right=True)

    plt.plot(x[i], isoTSmDensityBest_CCSn[i], color="red", label='${146}$Sm')
    plt.fill_between(x[i].flatten(), isoTSmDensityBest_CCSn[i], 0, color="re

    plt.xlabel('Isolation time (Myr)')
    plt.ylabel('Probability density')
    plt.legend(loc='upper right')

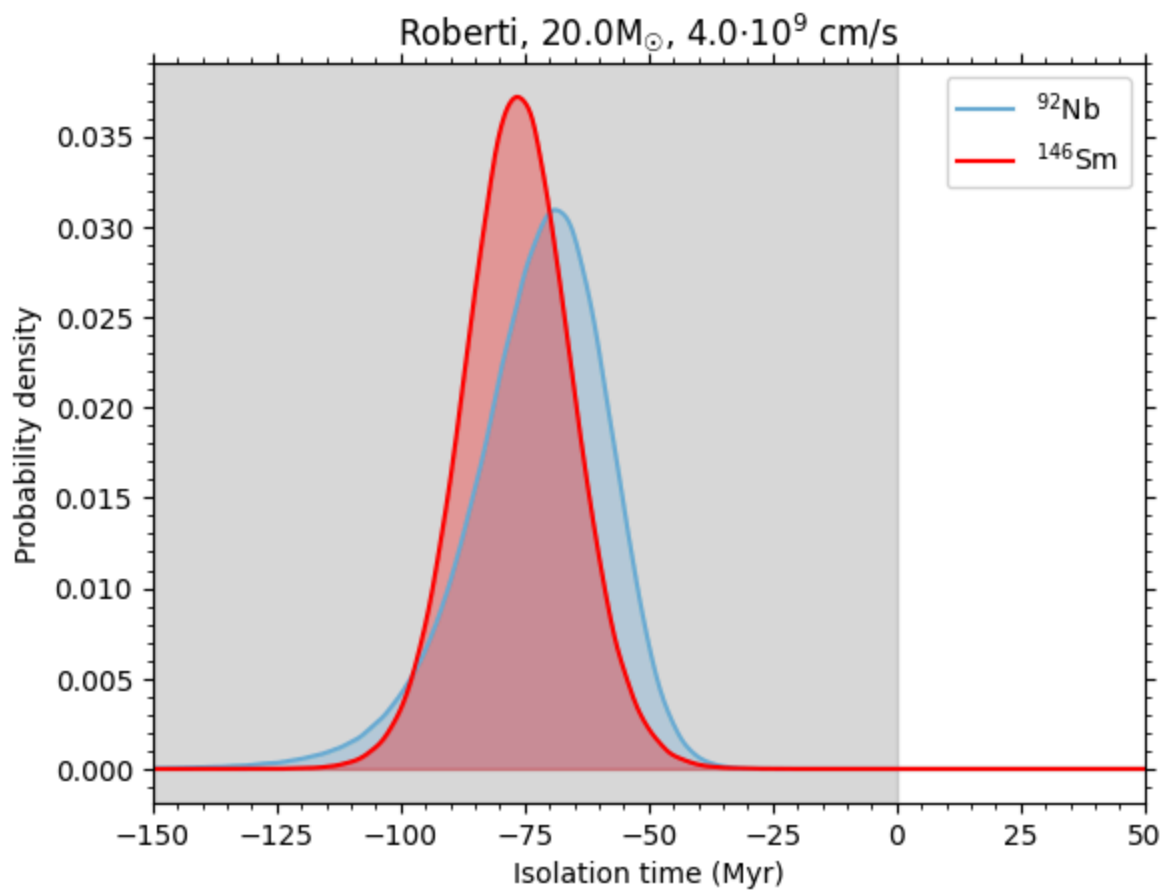
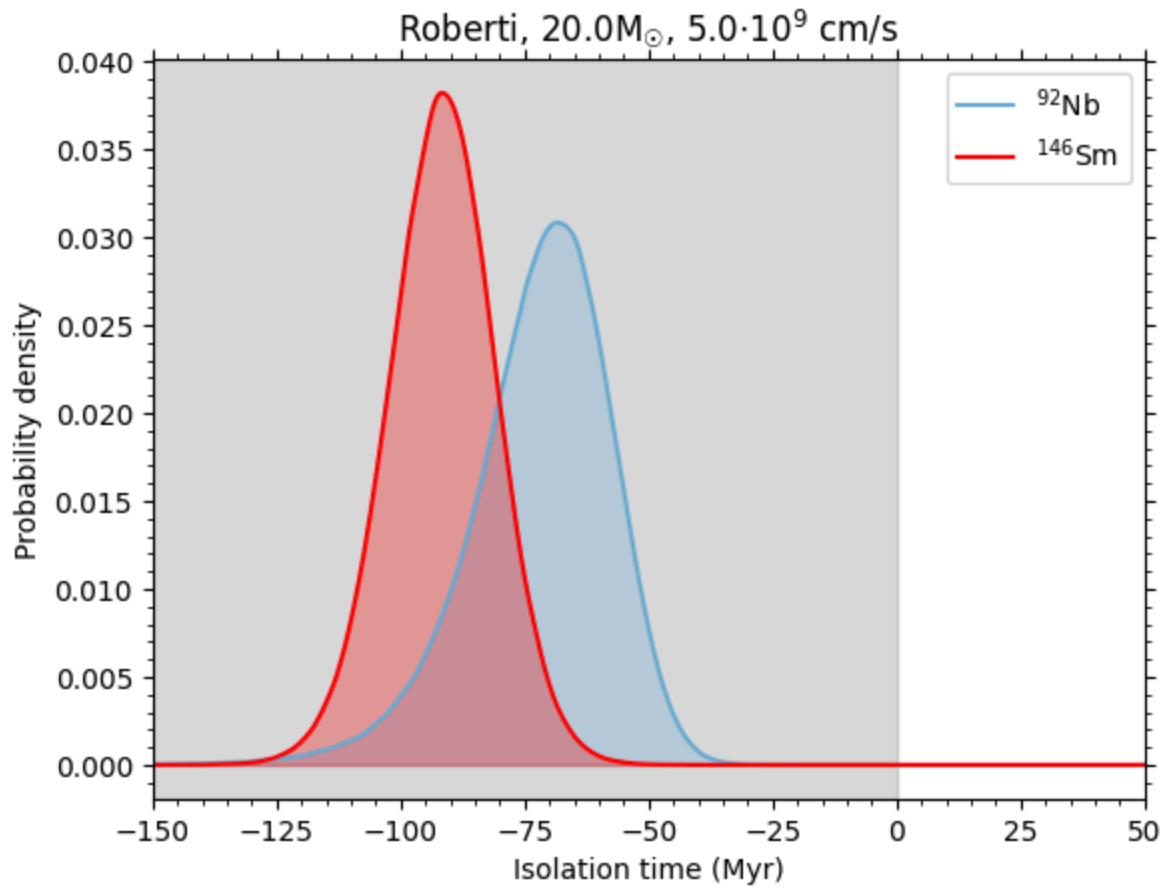
    #for the gray box
    plt.axvspan(xmin=x[i][0], xmax=0, color='gray', alpha=0.3, zorder=0)
    #plt.text(0.05, 0.95, "Isolation times < 0 \n(grey) are unphysical", tra

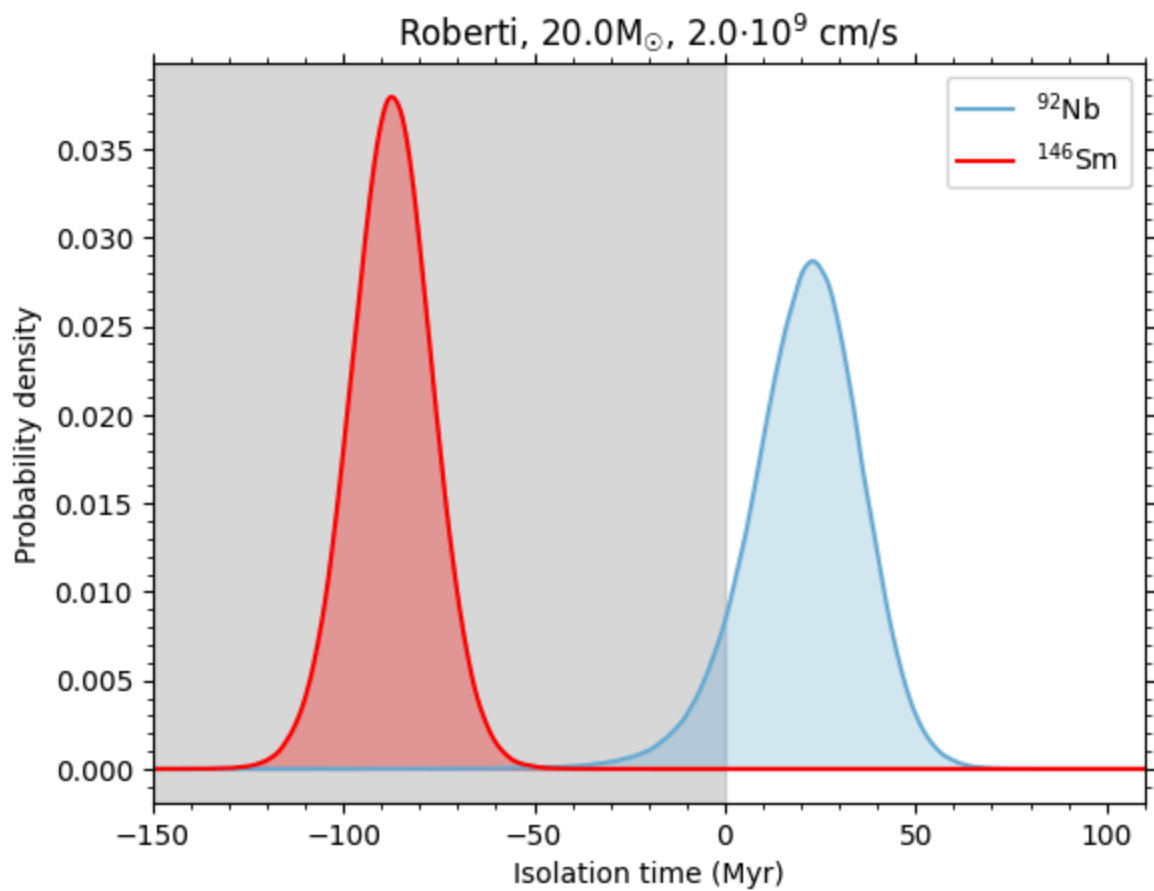
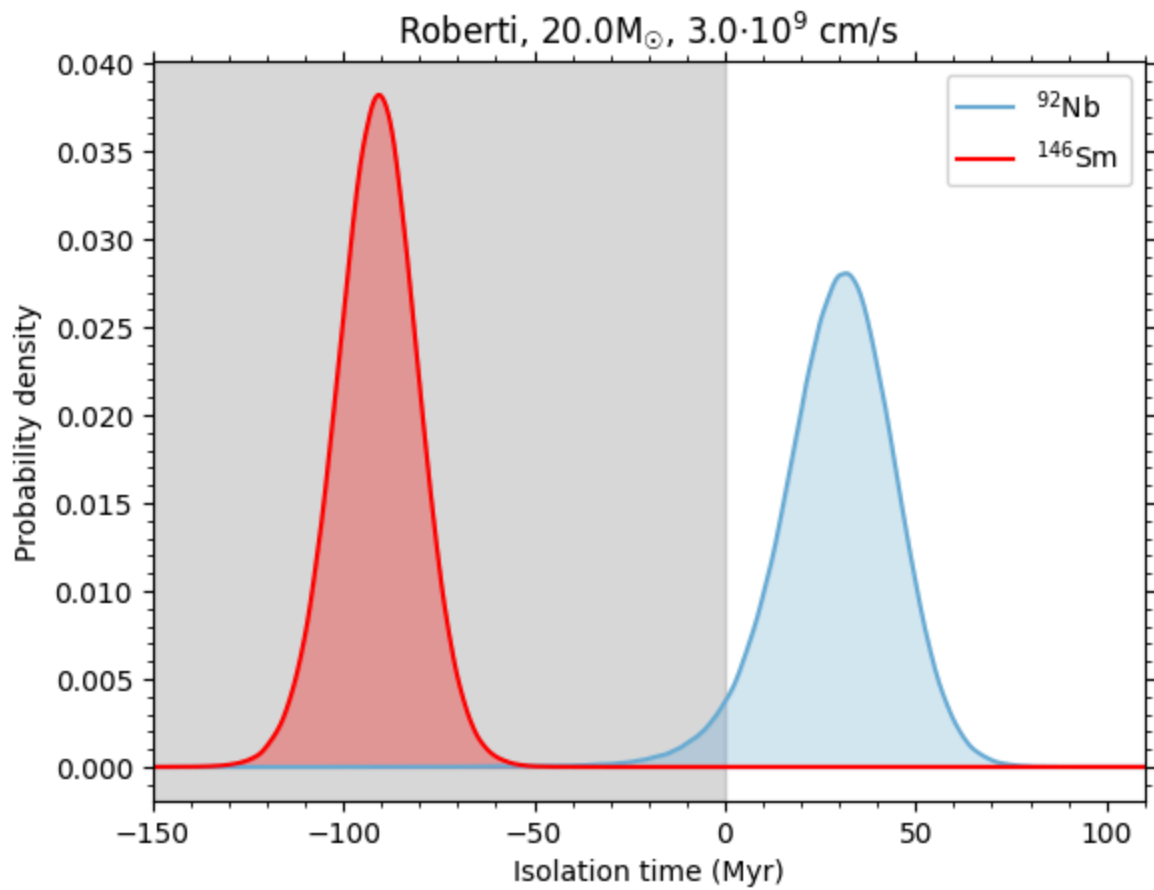
    plt.xlim(x[i][0], x[i][-1])

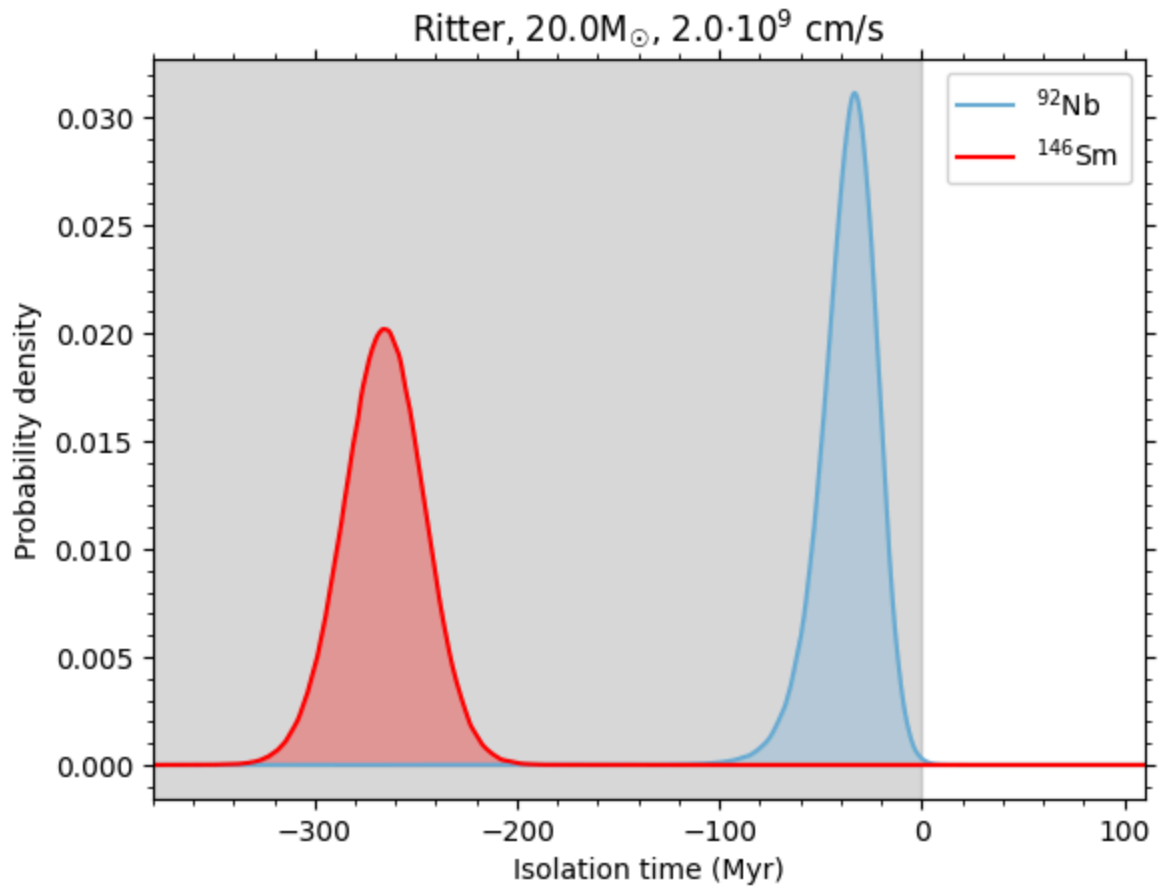
    plt.title(modelSet[i]+", "+str(mtot[i])+"M$_\odot$, "+str(shock[i])+"$\\cdot$

    plt.savefig("../figures/"+modelSet[i]+"_M"+str(int(mtot[i]))+"_V"+str(sh
    plt.show()

```







1σ uncertainty bands for ISS ratios

```
In [33]: #median, -, +
for i in range(np.size(prCCSnNb)):
    print(modelSet[i]+", "+str(mtot[i])+"M $\odot$ , "+str(shock[i])+" · 10 $^9$  cm/s")
    print("Nb: ",median_and_error(ratIsmNbCCSn[i]))
    print("Sm: ",median_and_error(ratIsmSmCCSn[i]))
    print()
```

Roberti, 20.0M_⊙, 5.0 · 10⁹ cm/s
 Nb: [7.752e-06 -1.925e-06 1.996e-06]
 Sm: [3.391e-03 -4.398e-04 4.694e-04]

Roberti, 20.0M_⊙, 4.0 · 10⁹ cm/s
 Nb: [7.656e-06 -1.901e-06 1.971e-06]
 Sm: [3.937e-03 -5.106e-04 5.450e-04]

Roberti, 20.0M_⊙, 3.0 · 10⁹ cm/s
 Nb: [5.871e-05 -1.458e-05 1.512e-05]
 Sm: [3.408e-03 -4.420e-04 4.718e-04]

Roberti, 20.0M_⊙, 2.0 · 10⁹ cm/s
 Nb: [4.953e-05 -1.230e-05 1.275e-05]
 Sm: [3.534e-03 -4.584e-04 4.893e-04]

Ritter, 20.0M_⊙, 2.0 · 10⁹ cm/s
 Nb: [1.576e-05 -3.915e-06 4.058e-06]
 Sm: [6.301e-04 -8.173e-05 8.723e-05]

1σ uncertainty bands for the Isolation Times

```
In [34]: #median, -, +
for i in range(np.size(prCCSnNb)):
    print(modelSet[i]+", "+str(mtot[i])+ "M⊙, "+str(shock[i])+ " · 109 cm/s")
    print("Nb: ",median_and_error(isoTNbCCSn[i]))
    print("Sm: ",median_and_error(isoTSmCCSn[i]))
    print()
```

Roberti, 20.0M_⊙, 5.0 · 10⁹ cm/s
 Nb: [-7.055e+01 -1.471e+01 1.194e+01]
 Sm: [-9.151e+01 -1.057e+01 1.032e+01]

Roberti, 20.0M_⊙, 4.0 · 10⁹ cm/s
 Nb: [-7.117e+01 -1.472e+01 1.195e+01]
 Sm: [-7.626e+01 -1.064e+01 1.078e+01]

Roberti, 20.0M_⊙, 3.0 · 10⁹ cm/s
 Nb: [3.019e+01 -1.512e+01 1.364e+01]
 Sm: [-9.099e+01 -1.057e+01 1.033e+01]

Roberti, 20.0M_⊙, 2.0 · 10⁹ cm/s
 Nb: [2.174e+01 -1.496e+01 1.335e+01]
 Sm: [-8.727e+01 -1.057e+01 1.042e+01]

Ritter, 20.0M_⊙, 2.0 · 10⁹ cm/s
 Nb: [-3.518e+01 -1.449e+01 1.205e+01]
 Sm: [-2.656e+02 -1.985e+01 1.951e+01]

References:

- Côté, Benoit et al. (Dec. 2019). "Stochastic Chemical Evolution of Radioactive Isotopes with a Monte Carlo Approach". In: ApJ 887.2, 213, p. 213. doi: [10.3847/1538-4357/ab5a88](https://doi.org/10.3847/1538-4357/ab5a88). arXiv: [1911.01457](https://arxiv.org/abs/1911.01457) [astro-ph.GA].
- Leckenby, G., Sidhu, R. S., Chen, R. J., Szányi, B., Dillmann, I., Gernhäuser, R., Glorius, J., Griffin, C., Litvinov, Y. A., Lugaro, M., Sanjari, S., & Yagüe López, A. (2024). Measurement of the bound-state beta decay of $^{205}\text{Tl}(81^+)$: analysis scripts and figures (1.0). Zenodo. <https://doi.org/10.5281/zenodo.11560338>
- Lugaro, M. et al. (Jan. 2016). "Origin of the p-process radionuclides ^{92}Nb and ^{146}Sm in the early solar system and inferences on the birth of the Sun". In: Proceedings of the National Academy of Science 113.13, pp. 907–912. doi: [10.1073/pnas.1519344113](https://doi.org/10.1073/pnas.1519344113). arXiv: [1601.05986](https://arxiv.org/abs/1601.05986) [astro-ph.SR].
- Tang, Shuling et al. (May 2025). "Examining evidence for a shorter $\text{Sm}^{146}\text{-Nd}^{142}$ chronology in the early solar system". In: PhRvC 111.5, L052801, p. L052801. doi: [10.1103/PhysRevC.111.L052801](https://doi.org/10.1103/PhysRevC.111.L052801).
- Ritter, C. et al. (Oct. 2018). "NuGrid stellar data set - II. Stellar yields from H to Bi for stellar models with $\text{MZAM } S = 1\text{-}25 \text{ M}_{\odot}$ and $Z = 0.0001\text{-}0.02$ ". In: MNRAS 480.1, pp. 538–571. doi: [10.1093/mnras/sty1729](https://doi.org/10.1093/mnras/sty1729). arXiv: [1709.08677](https://arxiv.org/abs/1709.08677) [astro-ph.SR].
- Roberti, L. et al. (June 2024). "The γ -process nucleosynthesis in core-collapse supernovae. II. Effect of the explosive recipe". In: A&A 686, L8, p. L8. doi: [10.1051/0004-6361/202449994](https://doi.org/10.1051/0004-6361/202449994). arXiv: [2405.07783](https://arxiv.org/abs/2405.07783) [astro-ph.SR].